

Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled “Core JDO” are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

Copyright (c) 2002 Sun Microsystems, Inc. All rights reserved.

FOR PRIVATE USE ONLY: This material is the property of Sun Microsystems, Inc. and is not to be sold, reproduced in any form by any means, or generally distributed without the prior written authorization of Sun Microsystems Inc and Prentice Hall.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdoobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdoobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdoobook@yahoogroups.com

" Few things are harder to put up with than the annoyance of a good example"
-Mark Twain

Case Study: The Core JDO Library

*T*he Core JDO Library application is an implementation of a couple of use cases to explain basic and advanced JDO features. It is also meant to explain concepts mentioned in the previous chapters such as two and n-tier applications. The chapter is broken into smaller sections that can be used as cook-book style recipes.

10.1 FILES, PACKAGES AND OBJECT MODEL

The files of the Core JDO library are in these directories:

src	Java sources
web	JSP sources and Web content
conf	Tomcat configuration
doc	JavaDoc output
classes	Compiler output

The project can be compiled by Ant (`build.xml`) and run either as a command-line application or as a web application. The Core JDO Library Java sources are split into three packages:

com.corejdo.casestudy...

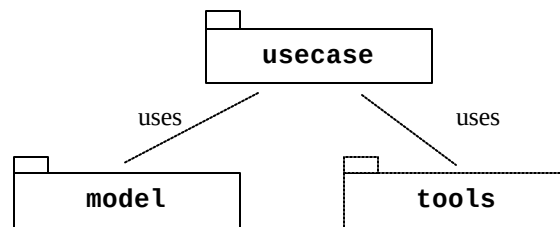


Figure 1: Core JDO library packages

The **usecase** package contains the business logic, serves as a testbed and supports command-line invocation. Every use case is coded in a single class and can be called either by a simple command-line interpreter or by external code. The pattern for this kind of implementation is called "Command": An instance of the **usecase** classes has a single **execute()** method and runs a single JDO transaction for that method. The benefit of this approach is a single point in the code, where transactions are begun, committed or rolled back. On the other hand exception handling and error processing can become quite uncomfortable.

The **model** package contains all persistence-capable classes, which define the class hierarchy shown in Figure 2.

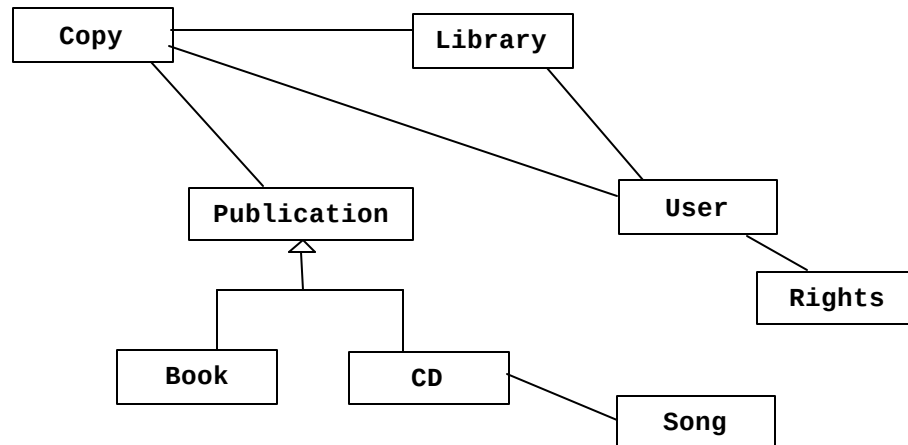


Figure 2: Persistent object model

10.2 PERSISTENT MODEL PACKAGE

The following sections look at each of the persistence capable classes.

10.2.1. Publication Class

This is the base class of books and CDs in the library.

```

package com.corejdo.casestudy.model;
import java.util.Date;
import java.util.Collection;
import javax.jdo.*;

/**
 * This class contains generic
 * information common to all sorts of publications.
 *
 * <BR><FONT Color="red"><B>Persistent</B></FONT>

```

```

    */
public abstract class Publication
{
    /**
     * The title of the publication.
     */
    public String title;
    /**
     * Date, when the publication was published.
     */
    public Date published;
    /**
     * Date, when the publication was first published.
     */
    public Date firstPublished;
    /**
     * A copyright text.
     */
    public String copyright;

    /**
     * The librarian who maintained the publication.
     */
    public User insertedBy;

    public Publication() {
    }

    public String toString()
    {
        return "\n Title: "+title+
            "\n Published: "+published+
            "\n First published: "+firstPublished+
            "\n Copyright: "+copyright;
    }

    public String toHTMLTitle(String linkPrefix)
    {
        return "<TD CLASS=pubTitle>"+title+"</TD>";
    }

    public String toHTMLRest(String linkPrefix)
    {
        return
            "<TD CLASS=pubPubFirst>"+firstPublished+"</TD>"+

```

```

        "<TD CLASS=pubCopyright>" + copyright + "</TD>";
    }

    public abstract String toHTML(String linkPrefix);
}

```

The `getCopies()` method encapsulates the JDO query to find all the `Copy` instances that reference a `Book` instance. Encapsulating a query in this way is a useful practice, although some may prefer to put these types of methods in a separate, associated class to avoid having any JDO dependent code in their persistence classes.

About toHTML() methods

The classes shown here include methods like `toHTML()`, `toHTMLTitle()` and `toHTMLRest()`, for the sake of simplicity and easily readable comprehensive examples.

Most real-world domain classes however would not include such presentation layer related methods and separate out view-related code into separate classes.

10.2.2. Book Class

This class represents a `Book`.

```

package com.corejdo.casestudy.model;

/**
 * This class contains data about books,
 * like author, International Book Number (ISBN).
 */

public class Book
    extends Publication
{
    /**
     * International book number.
     */
    public String ISBN;
    /**
     * Author of the book.
     */
    public String author;

    public Book() {

```

```

    }

    public String toString()
    {
        return super.toString()+
            "\n ISBN: "+ISBN+
            "\n Author: "+author;
    }
    public String toHTML(String linkPrefix)
    {
        return "<TR CLASS=Book>" + toHTMLTitle(linkPrefix) +
            "<TD CLASS=BookAuthor>" + author + "</TD>" +
            "<TD CLASS=BookISBN>" + ISBN + "</TD>" +
            toHTMLRest(linkPrefix) + "</TR>";
    }
}

```

10.2.3. CD Class

This class represents a CD, it is more complicated than the Book class since it provides more functionality. It contains a song list that is returned in `toString` and `toHTML`. It shows basic collection operations with `ArrayList`.

```

package com.corejdo.casestudy.model;

import java.util.*;

/**
 * <BR><FONT Color="red"><B>Persistent</B></FONT>
 */
public class CD
    extends Publication
{
    List songs = new ArrayList(); // contains Song objects
    String referenceNumber;         // CD reference number

    public CD(String referenceNumber)
    {
        this.referenceNumber = referenceNumber;
    }
}

```

```

private CD()
{
}

public void addSong(String composer,
                    String title, int seconds)
{
    songs.add(new Song(composer, title, seconds));
}

public String songTable()
{
    StringBuffer buf = new StringBuffer();
    buf.append("<TABLE class=song>\n");
    Iterator iter = songs.iterator();
    while (iter.hasNext()) {
        Song song = (Song)iter.next();
        buf.append("<TR class=song>");
        buf.append("<TD class=songTitle>");
        buf.append(song.title);
        buf.append("</TD>");
        buf.append("<TD class=songComposer>");
        buf.append(song.composer);
        buf.append("</TD>");
        buf.append("<TD class=songLength>");
        buf.append(song.length());
        buf.append("</TD>");
        buf.append("</TR>");
    }
    buf.append("</TABLE>\n");
    return buf.toString();
}

public String toString()
{
    StringBuffer buf = new StringBuffer();
    buf.append("CD: ");
    buf.append(this.referenceNumber);
    buf.append(super.toString());
    Iterator iter = songs.iterator();
    while (iter.hasNext()) {
        Song song = (Song)iter.next();
        buf.append(song);
        buf.append(", ");
    }
}

```

```

        return buf.toString();
    }

    public String toHTML(String linkPrefix)
    {
        return "<TR CLASS=CD>" + toHTMLTitle(linkPrefix) +
            "<TD VALIGN=TOP>" + songTable() + "</TD>" +
            "<TD VALIGN=TOP></TD>" +
            super.toHTMLRest(linkPrefix) + "</TR>";
    }
}

```

10.2.4. Copy Class

The Copy class represents a physical copy of a particular publication. For example, a book on a library shelf is represented as an instance of the Copy class. This class has a reference back to Publication. An alternative model to this would have used a collection of Copy instances in the Publication class. The benefit of this approach is that if there are a large number of instances, maintaining a collection of all of them can become problematic.

```

package com.corejdo.casestudy.model;

import java.util.Date;

/**
 * A copy is the instance of a publication in a
 * library. A copy can be located somewhere in the
 * library or it can be borrowed by someone.
 * Other states of a copy can be: missing, damaged or sold.
 * A copy cannot be reserved by someone, only
 * publications can be reserved.
 *
 * <BR><FONT Color="red"><B>Persistent</B></FONT>
 */

public class Copy
{
    /**
     * The publication of which this is the instance of.
     */
    public Publication copyOf;
}

```

```

    /**
     * This flag is set, when the borrowing time is over.
     */
    public boolean missing;
    /**
     * If the copy is damaged, this flag is set.
     */
    public boolean damaged;
    /**
     * If someone bought the copy, this flag is set.
     */
    public boolean sold;
    /**
     * Location of the copy in the library.
     */
    public String location;
    /**
     * Reference to the borrower. If set, this publication
instance is borrowed
     * by someone.
     */
    public User borrower;
    /**
     * Date when the copy was borrowed.
     * Depending on the user policies of the
     * library.
     */
    public Date borrowDate;
    /**
     * Date when the copy should be returned
     * to the library.
     */
    public Date returnDate;

    public Copy(Publication copyOf)
    {
        this.copyOf = copyOf;
        this.location = "entry";
    }

    private Copy()
    {
    }
}

```

10.2.5. User Class

The User class identifies a borrower and is used to define user rights or access restrictions.

```
package com.corejdo.casestudy.model;

/**
 * A user object represents a user of the
 * library. That may be either a
 * borrower or a librarian (or both).
 *
 * <BR><FONT Color="red"><B>Persistent</B></FONT>
 */
public class User
{
    public User()
    {
    }
    /**
     * Name of user.
     */
    private String firstName;
    /**
     * Name of user.
     */
    private String lastName;
    /**
     * Cleartext password.
     */
    private String password;
    /**
     * Internally used user id.
     */
    private String id;
    /**
     * The rights of the user.
     */
    private Rights rights;

    public String getFirstName()
    {
        return firstName;
    }
}
```

```

    }
    public void setFirstName(String firstName)
    {
        this.firstName = firstName;
    }
    public void setLastName(String lastName)
    {
        this.lastName = lastName;
    }
    public String getLastName()
    {
        return lastName;
    }
    public void setPassword(String password)
    {
        this.password = password;
    }
    public String getId()
    {
        return id;
    }
    public void setId(String id)
    {
        this.id = id;
    }

    public boolean checkPassword(String pw)
    {
        return pw.equals(password);
    }

    public Rights getRights()
    {
        if (rights == null) rights = new Rights();
        return rights;
    }

    public String toString()
    {
        return firstName+" "+lastName;
    }
}

```

10.2.6. Rights Class

The `Rights` class is an example of the "Enumeration Pattern". It wraps the `Right` class, which is a tri-state enumeration (`ALLOWED`, `DENIED`, `DEFAULT`). The `Rights` class uses attributes of type `Right` and is persistent.

```
package com.corejdo.casestudy.model;

/**
 * A transient tri-state value.
 * It may be set either to allow, deny or default.
 * <BR><B>not persistent</B>
 */
public final class Right
{
    boolean isAllowed(Right def)
    {
        if (this != DEFAULT) return this == ALLOWED;
        if (def == null || def == DEFAULT) return false;
        return def == ALLOWED;
    }

    boolean isDenied(Right def)
    {
        if (this != DEFAULT) return this == DENIED;
        if (def == null || def == DEFAULT) return true;
        return def == DENIED;
    }

    public static Right DENIED = new Right(-1);
    public static Right ALLOWED = new Right(1);
    public static Right DEFAULT = new Right(0);

    private int code;

    private Right(int code)
    {
        this.code = code;
    }
}
```

The `Rights` class:

```
package com.corejdo.casestudy.model;
```

```

import javax.jdo.InstanceCallbacks;
import javax.jdo.JDOHelper;
/**
 * Defines access restrictions for users and
 * user groups of the library.
 *
 * This is an example for a type mapping:
 * Instead of defining another persistent
 * class Right, the tri-state values
 * (allow, deny, default) are coded into plain
 * integer values, (+1, -1, 0 respectively).
 * For a better programming model, the
 * Right class is exposed to the application,
 * but stored internally as integer
 * values.
 *
 * <BR><FONT Color="red"><B>Persistent</B></FONT>
 */
public class Rights
    implements Cloneable
{
    /**
     * The right to add, remove or
     * modify users of the library.
     */
    private int _modifyUser;

    /**
     * The right to borrow something.
     */
    private int _borrow;

    /**
     * The right to manage publications.
     */
    private int _manage;

    /**
     * The constructor creates a default Rights object.
     * (An unauthorized user.)
     */

```

```

public Rights()
{
}

private int rightToInt(Right r)
{
    if (r == Right.ALLOWED) return 1;
    if (r == Right.DEFAULT) return 0;
    if (r == Right.DENIED) return -1;
    throw new RuntimeException("Illegal value: "+r);
}

private Right intToRight(int i)
{
    if (i == 1) return Right.ALLOWED;
    if (i == 0) return Right.DEFAULT;
    if (i == -1) return Right.DENIED;
    throw new RuntimeException("Illegal value: "+i);
}

public boolean canBorrow()
{
    return intToRight(
        _borrow).isAllowed(Right.ALLOWED);
}

public boolean canManage()
{
    return intToRight(_manage).isAllowed(Right.DENIED);
}

public boolean canModifyUser()
{
    return intToRight(
        _modifyUser).isAllowed(Right.DENIED);
}

public void setBorrow(Right r)
{
    _borrow = rightToInt(r);
}

public void setManage(Right r)
{
    _manage = rightToInt(r);
}

```

```

    public void setModifyUser(Right r)
    {
        _modifyUser = rightToInt(r);
    }

    public Rights copy()
    {
        try {
            int k = this._borrow;
            Rights r = (Rights)this.clone();
            return r;
        }
        catch (CloneNotSupportedException c) {
            throw new RuntimeException("Unexpected: "+c);
        }
    }

    public String toString()
    {
        String res =
            ( canBorrow() ? "borrow" : "noborrow")+
            ( canManage() ? ",manage" : ",nomanage")+
            ( canModifyUser() ? ",modify" : ",nomodify");
        return res;
    }
}

```

10.3 USECASE PACKAGE

This package contains the implementation of the application use-cases. It uses an abstract base class based on the command pattern that implements common methods for all transactions and defines an abstract execute method. Each use-case class extends this abstract class and implements the execute method.

10.3.1. AbstractUserCase Class

The base class for the use-cases implements the basic methods to get the PersistenceManagerFactory and PersistenceManager instances, begin and commit transactions and handle exceptions.

10.3.1.1. PersistenceManagerFactory Bootstrapping

As shown in earlier chapters PersistenceManagerFactory bootstrapping can be done using a singleton pattern and reading properties from a file to avoid hard coding connection or JDO implementation specific details.

```
/**
 * Creates or returns a JDO factory.
 * The properties are loaded
 * from a file.
 */
public static PersistenceManagerFactory
getPersistenceManagerFactory() {

    if (pmf == null) {

        String filename =
            System.getProperty("jdo.properties");

        if (filename == null) {

            throw new JDOFatalUserException(
                "System property 'jdo.properties' not defined");
        }

        Properties properties = new Properties();

        try {

            FileInputStream file =
                new FileInputStream(filename);

            properties.load(file);

            file.close();

            pmf = JDOHelper.getPersistenceManagerFactory(
                properties);
        }

        catch (java.io.IOException e) {

            throw new JDOFatalUserException(
                "Error reading '" + filename + "'",
                e);
        }
    }
}
```

```

    }
  }
  return pmf;
}

```

10.3.1.2. Obtaining the PersistenceManager

This method uses a singleton pattern to maintain a reference to a PersistenceManager instance for a given use-case instance.

```

/**
 * Creates or returns a PersistenceManager.
 * If a servlet helper is associated with
 * the current use case, a persistence manager
 * may be taken from the helper object.
 */
PersistenceManager getPM()
{
    if (currentPersistenceManager == null) {
        currentPersistenceManager =
            getPMF().getPersistenceManager();
    }
    return currentPersistenceManager;
}

```

10.3.1.3. Closing the PersistenceManager

This method releases the singleton PersistenceManager instance at the end of the use-case.

```

/**
 * Called by outside application management.
 *
 * In case of servlets, this method is called
 * on session timeout.
 * The command line version calls this method
 * at system exit.
 */
void close()
{
    if (currentPersistenceManager != null) {
        currentPersistenceManager.close();
        currentPersistenceManager = null;
    }
}

```

10.3.1.4. Execute Use-case

This method runs the use-case, it gets the PersistenceManager instance, begins a transaction and then calls the internal execute method to run the use-case. It then commits the transaction and handles any exceptions. If an exception is caught, the transaction is rolled back.

```
/**
 * This method creates a transaction context,
 * starts the transaction
 * and commits all work, after the <tt>_exceute</tt>
 * method has been
 * successfully invoked. If the sub-class throws
 * an exception, the transaction is aborted.
 */
public final void execute()
{
    if (!isAllowed())
        throw new UseCaseException("Not allowed.");
    getPM().currentTransaction().begin();
    boolean active = true;
    try {
        _execute();
        getPM().currentTransaction().commit();
        active = false;
    }
    finally {
        if (active) {
            try {
                getPM().currentTransaction().rollback();
                active = false;
            }
            catch (JDOUserException ex)
            {
                ex.printStackTrace();
            }
        }
    }
}
```

10.3.2. Adding Books Use-case

This first use-case is straight-forward, it creates a new `Book` instance and makes it persistent.

```
package com.corejdo.casestudy.usecase;

import com.corejdo.casestudy.model.Book;
import com.corejdo.casestudy.model.User;
import com.corejdo.casestudy.tools.CmdLine;

import java.util.Date;
import java.text.DateFormat;
import java.text.ParseException;

/**
 * This class adds a book to the library.
 * It implements a bean pattern, which can
 * be used simply in the JSP.
 */

public class AddBook
    extends AbstractUseCase
{
    /**
     * This is the book that gets inserted.
     * While it is transient, we
     * can set and get values directly.
     */
    Book book = new Book();

    public AddBook()
    {
    }

    public void setISBN(String ISBN)
    {
        book.ISBN = ISBN;
    }
    public void setAuthor(String author)
    {
        book.author = author;
    }
    public void setCopyright(String copyright)
    {
    }
}
```

```

        book.copyright = copyright;
    }
    public void setPublished(String published)
    {
        book.published = parseDate(published);
    }
    public void setFirstPublished(String firstPublished)
    {
        book.firstPublished = parseDate(firstPublished);
    }

    public void setTitle(String title)
    {
        book.title = title;
    }

    /**
     * Adds a book via command line.
     */
    public static void main(String[] args) {
        new AddBook().runCmdLine(args);
    }

    protected void runCmdLine(String[] args) {
        setTitle(CmdLine.getStringVar("Title", ""));
        setISBN(CmdLine.getStringVar("ISBN", ""));
        setAuthor(CmdLine.getStringVar(
            "Author", "Heiko Bobzin"));
        setCopyright(CmdLine.getStringVar(
            "Copyright", "(C) 2003 Prentice Hall"));
        String now = new Date().toString();
        setPublished(CmdLine.getStringVar(
            "Publishing date", now));
        setFirstPublished(CmdLine.getStringVar(
            "First publishing date", now));
        execute();
    }

    private Date parseDate(String d)
    {
        try {
            return DateFormat.getInstance().parse(d);
        }
        catch (ParseException e)
        {

```

```

        throw new RuntimeException("Illegal date: "+d);
    }
}

public boolean isAllowed()
{
    return getContext().userRights.canManage();
}

protected void _execute()
{
    // this is a really simple operation:
    User user = getUser();
    book.insertedBy = user;
    getPM().makePersistent(book);
    // set for next insert:
    book = new Book();
}

public String getFirstPublished()
{
    return new Date().toString();
}
public String getPublished()
{
    return new Date().toString();
}
}

```

10.4 BOOKOPERATION CLASS

The command pattern can be used to share common implementations of operations. This class provides an abstract implementation for common operations on books that other commands can share. Figure 3 shows the class diagram for a number of the use-cases.

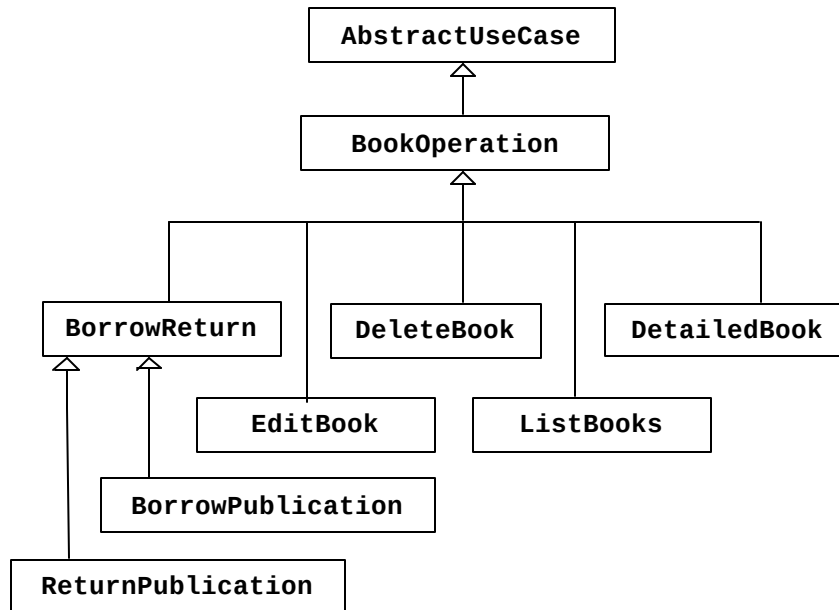


Figure 3: Inheriting from AbstractUseCase

This class implements a number of convenience methods that can be used to:

- Query generic fields by passing "field" and "field value"
- Return found Book instance as a subclass
- Ensure only a single instance result is returned, else error is thrown.
- Copies of a book can be found by "back pointer" or join query: returns all Copies that belong to a book.

```

package com.corejdo.casestudy.usecase;

import com.corejdo.casestudy.model.*;
import javax.jdo.*;
import java.util.*;

/**
 * Basic class of all Book operations.
 * This class can find a single book.
 */

public abstract class BookOperation extends AbstractUseCase
{

```

```

public BookOperation()
{
}

/**
 * Searches for a book that matches a string field
 * @param field name of field in the book class.
 * @param fieldValue value to match
 * @return a book
 */
protected Book findBook(String field,
                        String fieldValue)
{
    PersistenceManager pm = getPM();
    // find the book to update.
    Query q = pm.newQuery(Book.class, field+" == val");
    q.declareParameters("String val");
    q.setCandidates(pm.getExtent(Book.class,true));
    Collection result =
        (Collection)q.execute(fieldValue);
    if (result.size()<1) {
        throw new RuntimeException(
            field+" not found: "+fieldValue);
    } else if (result.size()>1) {
        throw new RuntimeException(
            field+" not unique: "+fieldValue);
    }
    Book book = (Book)result.iterator().next();
    return book;
}

protected final Book findBook()
{
    Book book = null;
    try {
        book = findBook("ISBN",this.findISBN);
    } catch (Exception e) {
        book = findBook("title",this.findTitle);
    }
    return book;
}

/**
 * Find all copies of a publication.

```

```

    */
    protected Copy[] findCopies(Publication publication)
    {
        PersistenceManager pm = getPM();
        // find all copies for a publication
        Query q = pm.newQuery(Copy.class, "copyOf == val");
        q.declareParameters(
            "com.corejdo.casestudy.model.Publication val");
        q.setCandidates(pm.getExtent(Copy.class, true));

        Collection result =
            (Collection)q.execute(publication);
        Copy copies[] = new Copy[result.size()];
        result.toArray(copies);
        return copies;
    }

    public void setFindTitle(String findTitle)
    {
        this.findTitle = findTitle;
    }
    public void setFindISBN(String findISBN)
    {
        this.findISBN = findISBN;
    }
    private String findTitle;
    private String findISBN;
}

```

10.4.1. Listing Books Use-case

This use-case shows how to display a list of all books or books that match a specified filter.

```

package com.corejdo.casestudy.usecase;

import com.corejdo.casestudy.tools.CmdLine;
import com.corejdo.casestudy.model.*;
import javax.jdo.*;
import java.util.*;

/**
 * This class lists books.
 */

```

```

public class ListBooks
    extends BookOperation
{
    int startNumber = 0;
    int count = 10;
    List result;

    String filterAuthor = "";
    Query query;
    Extent extent;

    /**
     * Sets the start offset. Since JDO does not
     * support a "seek" operation on
     * extents, we implemented our own by simply
     * calling iterator.next().
     * @param i
     */
    public void setStartNumber(int i)
    {
        startNumber = i;
    }

    /**
     * Sets the number of entries
     * returned by this operation.
     * @param i number of entries
     */
    public void setCount(int i)
    {
        count = i;
    }

    /**
     * This data object is used to return the results.
     */
    public class BookDTO
    {
        public final String title;
        public final String copyright;
        public final String ISBN;
        public final String author;

        BookDTO(Book b)
    }
}

```

```

    {
        this.title = b.title;
        this.copyright = b.copyright;
        this.ISBN = b.ISBN;
        this.author = b.author;
    }

    public String toString()
    {
        return "Title:  "+title+
            "\n   Author: "+author+
            "\n   ISBN:   "+ISBN+
            "\n   (C):    "+copyright;
    }
}

/**
 * Iterates over elements of the book Extent.
 * Called by the framework's execute() method.
 */
protected void _execute()
{
    result = new ArrayList();
    extent = getPM().getExtent(Book.class,true);
    query = null;
    Iterator iter = filter(extent);
    int i = this.startNumber;
    while (i-- > 0 && iter.hasNext()) iter.next();
    i = this.count;
    while (i-- > 0 && iter.hasNext()) {
        result.add(new BookDTO((Book)iter.next()));
    }
    if (query != null) {
        query.close(iter);
    } else {
        extent.close(iter);
    }
}

protected Iterator filter(Extent extent)
{
    if (filterAuthor != null) {
        query = getPM().newQuery(
            Book.class, "author == val");
        query.declareParameters(

```

```

        "java.lang.String val");
        query.setCandidates(extent);
        Collection result = (Collection)query.execute(
            filterAuthor);
        return result.iterator();
    } else {
        return extent.iterator();
    }
}

/**
 * Command line version.
 */
public static void main(String[] args) {
    new ListBooks().runCmdLine(args);
}

protected void run(String[] args) {
    setStartNumber(CmdLine.getIntVar("Start at",0));
    setCount(CmdLine.getIntVar("Count",10));
    execute();
    System.out.println("Result: ");
    Iterator iter = getResult();
    int i = 1;
    while (iter.hasNext()) {
        System.out.println(""+i+": "+iter.next());
        i++;
    }
}

/**
 * The resulting iterator returns BookDTO objects.
 */
public Iterator getResult()
{
    return result.iterator();
}
public int getCount()
{
    return count;
}
public int getStartNumber()
{
    return startNumber;
}

```

```

        public String getFilterAuthor()
        {
            return filterAuthor;
        }
        public void setFilterAuthor(String filterAuthor)
        {
            this.filterAuthor = filterAuthor;
        }
    }

```

10.4.2. Detailed Book Info Use-case

This use-case gets the detailed information about a particular book, it encapsulates a Book instance and provides an interface to get the detailed information from it without exposing the Book instance itself.

```

package com.corejdo.casestudy.usecase;

import com.corejdo.casestudy.model.*;
import com.corejdo.casestudy.tools.*;

/**
 * This class gets detailed information about a book.
 * It is a good example for information hiding.
 * The actual book data
 * in the persistent model Book class
 * uses a User field for the
 * borrower information, but unauthorized users must
 * not be able to see detailed information about borrowers.
 */
public class DetailedBook extends BookOperation
{
    public DetailedBook()
    {
    }

    public static void main(String[] args)
    {
        new DetailedBook().runCmdLine(args);
    }

    /**
     * Command line version of the book editor.
     */
}

```

```

protected void run(String[] args)
{
    setFindISBN(CmdLine.getStringVar(
        "Find ISBN", "928394-234-23444"));
    setFindTitle(CmdLine.getStringVar(
        "Find Title", ""));
    execute();
}

/**
 * First, looks up a book, then sets return value
 * for detailed book information.
 * The implementation of this
 * method looks quite simple because JDO does a
 * lot of things for us, like fetching other objects
 * on demand.
 */
protected void _execute()
{
    Book book = findBook();
    this.ISBN = book.ISBN;
    this.author = book.author;
    this.title = book.title;
    this.published = book.published.toString();
    this.firstPublished =
        book.firstPublished.toString();
    Copy copies[] = findCopies(book);
    availableCopies = 0;

    for (int i = 0; i < copies.length; i++) {
        if (copies[i].borrower == null)
            availableCopies++;
    }

    this.isBorrowed = availableCopies > 0;
}

public String getAuthor()
{
    return author;
}

public int getAvailableCopies()
{
    return availableCopies;
}

```

```

    public String getFirstPublished()
    {
        return firstPublished;
    }
    public String getISBN()
    {
        return ISBN;
    }
    public boolean isIsBorrowed()
    {
        return isBorrowed;
    }
    public String getPublished()
    {
        return published;
    }
    public String getTitle()
    {
        return title;
    }

    String ISBN;
    String author;
    String title;
    String published;
    String firstPublished;
    int availableCopies;
    boolean isBorrowed;
}

```

10.4.3. Edit Book Use-case

Similar to the previous use-case this one allows a particular Book instance to be modified. Again access to the actual Book instance is encapsulated by this class.

```

package com.corejdo.casestudy.usecase;

import com.corejdo.casestudy.model.Book;
import com.corejdo.casestudy.tools.CmdLine;

import javax.jdo.*;

```

```

import java.util.*;
import java.text.DateFormat;
import java.text.ParseException;

/**
 * This class is used to change properties of books.
 */

public class EditBook extends BookOperation
{
    public static void main(String[] args)
    {
        new EditBook().runCmdLine(args);
    }

    /**
     * Command line version of the book editor.
     */
    protected void run(String[] args)
    {
        setFindISBN(CmdLine.getStringVar(
            "Find ISBN", "928394-234-23444"));
        setFindTitle(CmdLine.getStringVar(
            "Find Title", ""));
        setISBN(CmdLine.getStringVar(
            "ISBN", "928394-234-23444"));
        setTitle(CmdLine.getStringVar(
            "Title", "new Title"));
        setAuthor(CmdLine.getStringVar(
            "Author", "H. Bobzin"));
        setCopyright(CmdLine.getStringVar(
            "Copyright", "(C) 2003 Prentice Hall"));
        execute();
    }

    /**
     * First, looks up a book, then updates the fields with
     * the values of this bean.
     */
    protected void _execute()
    {
        Book book = findBook();
        book.title = this.title;
    }
}

```

```

        book.author = this.author;
        book.copyright = this.copyright;
        book.ISBN = this.ISBN;
    }

    public void setISBN(String ISBN)
    {
        this.ISBN = ISBN;
    }
    public void setAuthor(String author)
    {
        this.author = author;
    }
    public void setTitle(String title)
    {
        this.title = title;
    }
    public void setCopyright(String copyright)
    {
        this.copyright = copyright;
    }
    private String ISBN;
    private String author;
    private String title;
    private String copyright;
}

```

10.4.4. Delete Book Use-case

This use-case simple deletes the specified Book instance. Interestingly it doesn't try to find all the Copy instances that reference this Book and delete them, although this would be a fairly straightforward addition. Instead the Copy instances are left as-is, now referencing a delete persistent object.

```

package com.corejdo.casestudy.usecase;
import com.corejdo.casestudy.tools.*;
import com.corejdo.casestudy.model.*;
import javax.jdo.*;
import java.util.*;

public class DeleteBook extends BookOperation
{
    public static void main(String[] args)

```

```

    {
        new DeleteBook().runCmdLine(args);
    }

    /**
     * Command line version.
     */
    protected void run(String[] args)
    {
        setFindISBN(CmdLine.getStringVar(
            "Find ISBN", "928394-234-23444"));
        setFindTitle(CmdLine.getStringVar(
            "Find Title", ""));
        execute();
    }

    public boolean isAllowed()
    {
        return getContext().userRights.canManage();
    }

    /**
     * First, looks up a book, then delete it.
     */
    protected void _execute()
    {
        Book book = findBook();
        PersistenceManager pm = getPM();
        pm.deletePersistent(book);
    }
}

```

10.4.5. BorrowReturn Class

This abstract class implements behavior common to borrowing or returning a book.

```

package com.corejdo.casestudy.usecase;

import com.corejdo.casestudy.model.*;
import com.corejdo.casestudy.tools.CmdLine;

import javax.jdo.*;

```

```

import java.util.*;
import java.text.DateFormat;
import java.text.ParseException;

/**
 * The common operations of "borrow"
 * and "return" are implemented in this class.
 */

public abstract class BorrowReturn extends BookOperation
{
    public boolean isAllowed()
    {
        if (!getContext().isLoggedIn()) return false;
        return getContext().userRights.canBorrow();
    }

    /**
     * First, looks up a book, then checks for a copy.
     * If copy exists, the book is borrowed or returned.
     */
    protected void _execute()
    {
        Book book = findBook();
        process(book);
    }

    protected abstract void process(Book book);

    public String getLocation()
    {
        return location;
    }
    public String getReturnDate()
    {
        return returnDate;
    }

    protected String returnDate;
    protected String location;
}

```

10.4.6. Borrow Use-case

This use-case tries to find available copies of a book and allocates one to the borrower.

```
package com.corejdo.casestudy.usecase;

...
    public final long borrowTime
        = 1000L*60L*10L; // ten minutes

    protected void process(Book book)
    {
        Copy copy[] = findCopies(book);
        for (int i = 0; i < copy.length; i++) {
            Copy c = copy[i];
            if (c.borrower == null) {
                c.borrower = getContext().user;
                c.borrowDate = new Date(); // now.
                c.returnDate = new Date(
                    c.borrowDate.getTime()+borrowTime);
                this.returnDate = c.returnDate.toString();
                this.location = c.location;
                return;
            }
        }
        throw new UseCaseException(
            "No copy available. Please ask clerk.");
    }
}
```

10.4.7. Return Use-case

Tries to find the copy of a book that was previously borrowed and returns it.

```
package com.corejdo.casestudy.usecase;

...
    protected void process(Book book)
    {
        Copy copy[] = findCopies(book);
        for (int i = 0; i < copy.length; i++) {
            Copy c = copy[i];
            if (c.borrower == getContext().user) {
```

```

        c.borrower = null;
        this.location = c.location;
        return ;
    }
}
throw new UseCaseException(
    "This book is not borrowed by "+
    getContext().user+". Please ask clerk.");
}

```

10.5 PUTTING THINGS TOGETHER

The project can be compiled by ant (<http://jakarta.apache.org>) but has to be modified slightly to use other JDO implementations. A `build.xml` file can be found in the top level directory.

10.5.1. XML Meta data

The `model.jdo` meta data file is located in the `src/com/corejdo/casestudy` directory and follows JDO 1.0 naming convention.

```

<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE jdo PUBLIC "-//Sun Microsystems, Inc.//DTD Java
Data Objects
Metadata 1.0//EN" "http://java.sun.com/dtd/jdo\_1\_0.dtd">
<jdo>
    <package name="com.corejdo.casestudy.model">
        <class name="Book">
        </class>
        <class name="CD">
        </class>
        <class name="Copy">
        </class>
        <class name="Library">
        </class>
        <class name="Publication">
        </class>
        <class name="Rights">
        </class>
    </package>

```

```

        <class name="Song">
        </class>
        <class name="User">
        </class>
    </package>
</jdo>

```

10.5.2. Running in command line mode

The command line application can be started by an `ant` target as well:

```
$ ant run
```

```

Use case [ListBooks]: Type name of use case here!
Start at [0]:
Count [10]:

```

```

Result:
1: Title:  Travel Guide: Sweden
   Author: Joseph, Bert
   ISBN:   7-7333-4727-6
   (C):    O'Renny
...

```

The data store can be filled with random data by the `FillDatabase` command.

For some operations you have to login. The initial user name is "su" and the password is "su" as well. Here is how to add new users:

```

Use case [ListBooks]: Login
UserId []: su
Password []: su
Use case [Login]: EditUser
Find User Id [hb]: smith
User not found.
Add [false]: true
First name: John
Last name: Smith
Password: js
May borrow [false]: true

```

```

May manage publications [false]: true
May manage users [false]: true
Use case [EditUser]: _

```

10.5.3. Running the servlets

Figure 4 shows a screenshot of a web page, that is generated from the `ListBooks.jsp` code. Again, `ant` can be used to start the `tomcat` web server. The pages are compiled on demand. Please make sure that the Java SDK is set in the `classpath`, it is needed by the `tomcat` JSP engine.

\$ `ant servlet` starts the server.

CoreJDO
Library

List books
List books and CDs
Get status
Login...

Maintenance operations:
Add a book
Modify user
Recreate database

List books

0 Back

Next 10

Title	Author	
Travel Guide: Sweden	Joseph, Bert	Details
Welcome to France	Oleary, Sarah	Details
Biking through Barbados	Nevly, Eliza	Details
How Barbados Changed My Life	Kensington, Petra	Details
Sailing around Hungary	Idola, Petra	Details
Welcome to Denmark	Oleary, Denise	Details
How Venezuela Changed My Life	Andon, Ulf	Details
Welcome to Norway	Morris, Otilie	Details
How Africa Changed My Life	Larson, Lee	Details
How To Get a Job in Venezuela	Bogarek, Francis	Details

Figure 4: A Running servlet